

Beszélgetés a Geminivel

szia

Szia! Miben lehetek a segítségére ma?

Oldjuk meg ezt a zh-t csinálom közbe intellij-ban tomcat 10-es vezió illetve mavent és mariadb-t használok, a security-t jaas.config-al listnerrel szoktuk megadni plusz authentication module (így szoktak általában kinézni a struktúrák amiket küldtem screenshotokba)

Szia! Szívesen segítek végigmenni ezen a pótpótZH feladaton. Mivel ez egy elég komplex, két külön alkalmazásból álló vizsga, a leghatékonyabb az lesz, ha lépésről lépésre építjük fel a projektet, pont úgy, ahogy a képernyőképeken is látható struktúrát megszoktad.

PDF+ 1

Kezdjük az alapok és a szerver konfigurációjának tisztázásával, majd felvázolom a két alkalmazás architektúráját.

1. Adatbázis és Tomcat JNDI beállítások

Mivel mindkét alkalmazás adatbázist használ, az első lépés a Tomcat konfigurálása. A leírás alapján az alkalmazásszerveren a context.xml fájlban (vagy a projektjeid META-INF/context.xml fájljában) fel kell venni a két adatbázis erőforrást.

PDF

Íme a pontos XML konfiguráció a feladateleírás alapján összeállítva (figyelve az idézőjelekre):

XML

<Context>

```
<Resource name="jdbc/PotPotZH_university_MariaDB"
    type="javax.sql.DataSource"
    driverClassName="org.mariadb.jdbc.Driver"
    factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
    url="jdbc:mariadb://localhost:3306/potpotzh_university_database"
    username="potpotzh_university_admin_user"
    password="PotPotZH2026Password111"
```

```
maxActive="20" maxIdle="10" maxWait="-1" />
```

```
<Resource name="jdbc/PotPotZH_security_MariaDB"
    type="javax.sql.DataSource"
    driverClassName="org.mariadb.jdbc.Driver"
    factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
    url="jdbc:mariadb://localhost:3306/potpotzh_security_database"
    username="potpotzh_security_admin_user"
    password="PotPotZH2026SecurityPassword111"
    maxActive="20" maxIdle="10" maxWait="-1" />
```

</Context>

Megjegyzés: Ellenőrizd, hogy a mariadb-java-client-3.1.3.jar fájl be van-e másolva a Tomcat lib könyvtárába, különben nem fog tudni csatlakozni az adatbázishoz.

PDF

2. Első alkalmazás: potpotzh-feladat

Ez a backend/core alkalmazás. Ennek a feladata az adatok közvetlen kezelése és kiajánlása.

PDF

- **Adatbázis műveletek:** Ez az alkalmazás kommunikál a potpotzh_university_database adatbázissal. Tartalmaznia kell egy Repository réteget a hallgatók lekérdezéséhez (lista, azonosító alapján), létrehozásához és törléséhez.

PDF

- **Service & Controller:** Szükség lesz egy Service rétegre és egy Controller rétegre, ami API endpointokat biztosít.

PDF

- **SOAP WebService:** Egy webservice-t is létre kell hoznod, amin keresztül a másik alkalmazás eléri az adatbázis műveleteket.

PDF

- **Megjelenítés:** Egy sima Servletet kell írnod (nem JSP!), ami kilistázza a hallgatókat név szerinti ABC sorrendben.

PDF

- **Security:** Ebben az alkalmazásban nincs biztonsági réteg (security).

PDF

3. Második alkalmazás: potpotzh-jsp-feladat

Ez a frontend/kliens alkalmazás, ahol a felhasználók belépnek és interakcióba lépnek a rendszerrel.

PDF

- **Adatelérés:** Az adatokat nem közvetlenül az adatbázisból, hanem az első alkalmazás SOAP webservice-én keresztül éri el és módosítja.

PDF

- **JSP Felületek:** Itt már JSP-t kell használni a listázáshoz, ahol gombok vannak a törléshez és az XML konzolos logoláshoz. Szükség van egy felületre az új hallgató rögzítéséhez is.

PDF+ 3

- **Egyedi Tag Library (JSTL):** A listázó gomb alatt lennie kell egy gombnak, ami egy custom tag library segítségével (service hívás nélkül) kiírja a legjobb átlaggal rendelkező hallgató nevét és szakját.

PDF

- **Navigáció:** A listázó és rögzítő oldalakon implementálni kell navigációt (page) és egy kijelentkezés linket.

PDF

- **JAAS Security:** Itt kell alkalmaznod a screenshotokon is mutatott JAAS struktúrát (Listener, Authentication Module, jaas.config). Ez egy FORM alapú autentikáció lesz.

PDF

- **Szerepkörök:** creator (létrehozó) az új hallgatók rögzítéséhez , és remover (törlő) a törléshez.

PDF+ 1

- **Nyilvános/Közös végpontok:** A listázás, az XML generálás és a legjobb hallgató funkció mindkét szerepkörnek elérhető. A webservice maga pedig bejelentkezés nélkül is hozzáférhető kell legyen.

PDF+ 1

Tomcat 10 Fontos Szabály: Mivel Tomcat 10-et használsz, az importoknál figyelj arra, hogy a javax.servlet.* helyett mindenhol jakarta.servlet.* csomagokat használj, a webservice-nél pedig jakarta.jws.* csomagot!

Melyik résszel vagy alkalmazással szeretnéd elkezdni a kódolást (pl. a potpotzh-feladat entitásainak és repository-jának felépítésével, vagy a Tomcat projekt alapjainak generálásával)?

menjunk sorba

Rendben, akkor haladjunk teljesen az elejétől, logikusan felépítve. Kezdjük a 0. feladattal (Git) és az első, potpotzh-feladat nevű alkalmazás alapjaival.

PDF+ 1

1. Lépés: Git és alapvető struktúra (0. feladat)

A leírás kifejezetten kéri, hogy hozz létre egy git repository-t, és egy develop branchen dolgozz.

PDF

Nyiss egy terminált a letöltött és kicsomagolt mappában (ahol majd a zh mappa is lesz), és add ki ezeket a parancsokat:

Bash

git init

git checkout -b develop

git add .

git commit -m "Initial commit - kiinduló fájlok hozzáadása"

Fontos: A ZH során minden logikai egység (feladat) után csinálj egy külön commitot beszédes üzenettel, mert az egyben commitált ZH 0 pontot ér.

PDF+ 1

2. Lépés: Maven pom.xml beállítása a potpotzh-feladat-hoz

Mivel Tomcat 10-et használsz, elengedhetetlen a jakarta.* névterek használata. Hozd létre a potpotzh-feladat webalkalmazást, és a pom.xml-be a következő dependency-k kellenek a Servlet API-hoz, a MariaDB-hez és később a SOAP weboldalhoz:

XML

<dependencies>

<dependency>

```

    <groupId>jakarta.servlet</groupId>

    <artifactId>jakarta.servlet-api</artifactId>

    <version>6.0.0</version>

    <scope>provided</scope>
</dependency>
<dependency>

    <groupId>org.mariadb.jdbc</groupId>

    <artifactId>mariadb-java-client</artifactId>

    <version>3.1.3</version>
</dependency>
<dependency>

    <groupId>jakarta.jws</groupId>

    <artifactId>jakarta.jws-api</artifactId>

    <version>3.0.0</version>

    <scope>provided</scope>
</dependency>
</dependencies>

```

3. Lépés: Entitás létrehozása (Hallgato.java)

Az init_university_db.sql alapján a hallgato tábla mezőit kell leképeznünk egy Java osztályba. Hozd létre a hu.pte.mik.prog4.potzh_2026.entity csomagot.

Java

```
package hu.pte.mik.prog4.potzh_2026.entity;
```

```

public class Hallgato {

    private int id;

    private String hallgatoNev;

    private String szak;

    private int felev;

```

```

private double egyetemiAtlag; // A DB-ben decimal(3,2)

public Hallgato() {
}

public Hallgato(int id, String hallgatoNev, String szak, int felev, double egyetemiAtlag) {
    this.id = id;
    this.hallgatoNev = hallgatoNev;
    this.szak = szak;
    this.felev = felev;
    this.egyetemiAtlag = egyetemiAtlag;
}

// Ide generáld le a Gettereket és Settereket az IDE-vel!
}

```

4. Lépés: Repository alapok és adatbázis kapcsolat

Az első alkalmazásban létre kell hozni a repository réteget, ami a potpotzh_university_database adatbázissal kommunikál. Hozd létre a hu.pte.mik.prog4.potzh_2026.repository csomagot, benne a HallgatoRepository osztállyal.

PDF

Itt JNDI lookup-ot használunk, hogy a Tomcat context.xml-ben megadott jdbc/PotPotZH_university_MariaDB resource-on keresztül kapjunk kapcsolatot.

PDF

Java

```

package hu.pte.mik.prog4.potzh_2026.repository;

import hu.pte.mik.prog4.potzh_2026.entity.Hallgato;
import javax.naming.Context;

```

```

import javax.naming.InitialContext;

import javax.sql.DataSource;

import java.sql.Connection;

public class HallgatoRepository {

    // Adatbázis kapcsolat megszerzése a Tomcat connection pool-ból

    private Connection getConnection() throws Exception {

        Context initContext = new InitialContext();

        Context envContext = (Context) initContext.lookup("java:/comp/env");

        DataSource ds = (DataSource)
envContext.lookup("jdbc/PotPotZH_university_MariaDB");

        return ds.getConnection();

    }

    // A feladat kéri a következőket: lekérdezni listát, azonosító alapján keresni, létrehozni
és törölni.

}

```

Ezek az alapok megvannak a projektedben? Ha igen, megírhatom a HallgatoRepository pontos SQL metódusait (findAll, findById, save, delete), hogy aztán rátérhessünk a Service és Controller rétegre?

tovább

Folytassuk a repository metódusaival, majd építsük rá a service és a controller réteget.

PDF

5. Lépés: A HallgatoRepository befejezése

A feladat megköveteli, hogy lehessen listázni, azonosító alapján keresni, létrehozni és törölni. Egészítsd ki a HallgatoRepository osztályt a következő metódusokkal. A try-with-resources blokkok gondoskodnak a kapcsolatok és a statementek megfelelő lezárásáról.

PDF+ 1

Java

```
package hu.pte.mik.prog4.potzh_2026.repository;
```

```
import hu.pte.mik.prog4.potzh_2026.entity.Hallgato;
```

```
import javax.naming.Context;
```

```
import javax.naming.InitialContext;
```

```
import javax.sql.DataSource;
```

```
import java.sql.Connection;
```

```
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class HallgatoRepository {
```

```
    private Connection getConnection() throws Exception {
```

```
        Context initContext = new InitialContext();
```

```
        Context envContext = (Context) initContext.lookup("java:/comp/env");
```

```
        DataSource ds = (DataSource)
```

```
envContext.lookup("jdbc/PotPotZH_university_MariaDB");
```

```
        return ds.getConnection();
```

```
    }
```

```
    public List<Hallgato> findAll() {
```

```
        List<Hallgato> hallgatok = new ArrayList<>();
```

```
        String sql = "SELECT * FROM hallgato ORDER BY hallgato_nev ASC";
```



```

try (Connection conn = getConnection();

    PreparedStatement ps = conn.prepareStatement(sql);

    ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        hallgatok.add(new Hallgato(
            rs.getInt("ID"),
            rs.getString("hallgato_nev"),
            rs.getString("szak"),
            rs.getInt("felev"),
            rs.getDouble("egyetemi_atlag")
        ));
    }
} catch (Exception e) {
    e.printStackTrace();
}

return hallgatok;
}

```

```

public Hallgato findById(int id) {

    String sql = "SELECT * FROM hallgato WHERE ID = ?";

    try (Connection conn = getConnection();

        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, id);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                return new Hallgato(

```

```

        rs.getInt("ID"),
        rs.getString("hallgato_nev"),
        rs.getString("szak"),
        rs.getInt("felev"),
        rs.getDouble("egyetemi_atlag")
    );
}
}
} catch (Exception e) {
    e.printStackTrace();
}
return null;
}

```

```

public void save(Hallgato hallgato) {

    String sql = "INSERT INTO hallgato (hallgato_nev, szak, felev, egyetemi_atlag) VALUES
    (?, ?, ?, ?)";

    try (Connection conn = getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, hallgato.getHallgatoNev());
        ps.setString(2, hallgato.getSzak());
        ps.setInt(3, hallgato.getFelev());
        ps.setDouble(4, hallgato.getEgyetemiAtlag());
        ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

    }

    public void delete(int id) {
        String sql = "DELETE FROM hallgato WHERE ID = ?";
        try (Connection conn = getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, id);
            ps.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

6. Lépés: A Service réteg létrehozása

A leírás szerint a service réteg minden repository metódushoz kell, hogy tartalmazzon egy hívást. Hozd létre a `hu.pte.mik.prog4.potzh_2026.service` csomagot, benne a `HallgatoService` osztállyal.

PDF

Java

```

package hu.pte.mik.prog4.potzh_2026.service;

import hu.pte.mik.prog4.potzh_2026.entity.Hallgato;
import hu.pte.mik.prog4.potzh_2026.repository.HallgatoRepository;

import java.util.List;

public class HallgatoService {

```

```

private final HallgatoRepository repository;

public HallgatoService() {
    this.repository = new HallgatoRepository();
}

public List<Hallgato> getAllHallgato() {
    return repository.findAll();
}

public Hallgato getHallgatoById(int id) {
    return repository.findById(id);
}

public void saveHallgato(Hallgato hallgato) {
    repository.save(hallgato);
}

public void deleteHallgato(int id) {
    repository.delete(id);
}
}

```

7. Lépés: SOAP WebService létrehozása (SoapHallgatoService)

A feladat kéri egy webservice elkészítését, amin keresztül a másik alkalmazás eléri az adatbázis műveleteket. Ehhez a Tomcat 10-ben a jakarta.jws annotációkat kell használni. Hozd létre a hu.pte.mik.prog4.potzh_2026.ws csomagot.

PDF+ 2

Java

```
package hu.pte.mik.prog4.potzh_2026.ws;
```

```
import hu.pte.mik.prog4.potzh_2026.entity.Hallgato;
import hu.pte.mik.prog4.potzh_2026.service.HallgatoService;
```

```
import jakarta.jws.WebMethod;
import jakarta.jws.WebParam;
import jakarta.jws.WebService;
import java.util.List;
```

```
@WebService(serviceName = "HallgatoDataService")
```

```
public class SoapHallgatoService {
```

```
    private final HallgatoService service = new HallgatoService();
```

```
    @WebMethod
```

```
    public List<Hallgato> getHallgatok() {
```

```
        return service.getAllHallgato();
```

```
    }
```

```
    @WebMethod
```

```
    public Hallgato getHallgato(@WebParam(name = "id") int id) {
```

```
        return service.getHallgatoById(id);
```

```
    }
```

```
    @WebMethod
```

```
    public void createHallgato(
```

```
        @WebParam(name = "nev") String nev,
```

```
        @WebParam(name = "szak") String szak,
```

```

        @WebParam(name = "felev") int felev,

        @WebParam(name = "atlag") double atlag) {

    Hallgato h = new Hallgato(0, nev, szak, felev, atlag);

    service.saveHallgato(h);

}

@WebMethod

public void deleteHallgato(@WebParam(name = "id") int id) {

    service.deleteHallgato(id);

}
}

```

Fontos: Ahhoz, hogy ez a webservice működjön, a WEB-INF/sun-jaxws.xml fájlban (ahogy a screenshotodon is látszik az előző munkádból) regisztrálni kell az endpointot, illetve a web.xml-ben is be kell kötni a WSServlet-et.

Jöhet a sun-jaxws.xml és a web.xml konfigurációja, hogy a webservice életre keljen, vagy előbb megírjuk a sima listázó Servletet (ami nem JSP) ehhez az első alkalmazáshoz?

PDF+ 1

tovább

Szuper, haladjunk tovább! Most bekötjük a webservice-t a konfigurációs fájlokba, majd megírjuk a kötelező listázó Servletet és az API Controllert az első alkalmazáshoz.

8. Lépés: JAX-WS és Web konfiguráció (sun-jaxws.xml és web.xml)

Ahhoz, hogy a SOAP webszolgáltatásod elérhető legyen a Tomcat szerveren, konfigurálni kell a végpontokat a WEB-INF mappában.

Hozd létre a src/main/webapp/WEB-INF/sun-jaxws.xml fájlt:

XML

```

<?xml version="1.0" encoding="UTF-8"?>

<endpoints xmlns="https://jakarta.ee/xml/ns/jax-ws/ri/runtime" version="3.0">

    <endpoint name="SoapHallgatoService"

        implementation="hu.pte.mik.prog4.potzh_2026.ws.SoapHallgatoService"

```

```
url-pattern="/ws/hallgatodata"/>
```

```
</endpoints>
```

Frissítsd vagy hozd létre a src/main/webapp/WEB-INF/web.xml fájlt, hogy a JAX-WS Servlet elinduljon, és hivatkozzunk a JNDI adatbázis resource-ra:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee  
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
```

```
version="6.0">
```

```
<listener>
```

```
<listener-  
class>com.sun.xml.ws.transport.http.servlet.WSServletContextListener</listener-  
class>
```

```
</listener>
```

```
<servlet>
```

```
<servlet-name>JAXWSServlet</servlet-name>
```

```
<servlet-class>com.sun.xml.ws.transport.http.servlet.WSServlet</servlet-class>
```

```
<load-on-startup>1</load-on-startup>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
<servlet-name>JAXWSServlet</servlet-name>
```

```
<url-pattern>/ws/hallgatodata</url-pattern>
```

```
</servlet-mapping>
```

```
<resource-ref>

  <description>DB Connection</description>

  <res-ref-name>jdbc/PotPotZH_university_MariaDB</res-ref-name>

  <res-type>javax.sql.DataSource</res-type>

  <res-auth>Container</res-auth>

</resource-ref>
```

```
</web-app>
```

(Megjegyzés: Ha a maven buildnél a webservice futtatásához hiányozna a runtime, a pom.xml-be esetleg be kell húzni a com.sun.xml.ws:jaxws-rt függőséget is, de csak ha hibát dob a Tomcat induláskor!)

9. Lépés: A Listázó Servlet (NEM JSP!)

A feladat kéri: "Ennek az alkalmazásnak van egy felülete, ami listázza az adatbázis táblában található adatokat, a felületet a servlet rendeli ki (nem jsp!) a hallgatók neve szerinti abc sorrendben."

PDF

Hozd létre a hu.pte.mik.prog4.potzh_2026.servlet csomagot, és benne a HallgatoListServlet osztályt. Mivel a Repository-ban a SELECT * FROM hallgato ORDER BY hallgato_nev ASC lekérdezést írtuk meg, a sorrend már automatikusan jó lesz.

Java

```
package hu.pte.mik.prog4.potzh_2026.servlet;
```

```
import hu.pte.mik.prog4.potzh_2026.entity.Hallgato;
```

```
import hu.pte.mik.prog4.potzh_2026.service.HallgatoService;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```



```
import java.io.IOException;

import java.io.PrintWriter;

import java.util.List;

@WebServlet("/list")

public class HallgatoListServlet extends HttpServlet {

    private final HallgatoService service = new HallgatoService();

    @Override

    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

        resp.setContentType("text/html;charset=UTF-8");

        List<Hallgato> hallgatok = service.getAllHallgato();

        try (PrintWriter out = resp.getWriter()) {

            out.println("<!DOCTYPE html>");

            out.println("<html>");

            out.println("<head><title>Hallgatók Listája</title></head>");

            out.println("<body>");

            out.println("<h2>Hallgatók Listája (ABC sorrend)</h2>");

            out.println("<table border='1'>");

            out.println("<tr><th>ID</th><th>Név</th><th>Szak</th><th>Félév</th><th>Átlag</th><
/tr>");

            for (Hallgato h : hallgatok) {

                out.println("<tr>");

                out.println("<td>" + h.getId() + "</td>");
```

```

        out.println("<td>" + h.getHallgatoNev() + "</td>");
        out.println("<td>" + h.getSzak() + "</td>");
        out.println("<td>" + h.getFelev() + "</td>");
        out.println("<td>" + h.getEgyetemiAtlag() + "</td>");
        out.println("</tr>");
    }

    out.println("</table>");
    out.println("</body>");
    out.println("</html>");
}
}
}

```

10. Lépés: API Controller endpointok

A leírás kitér rá: "Ennek az alkalmazásnak van egy controller rétege, ami api endpointokat tartalmaz a service rétegben található metódusokhoz."

PDF+ 1

Hozd létre a `hu.pte.mik.prog4.potzh_2026.api.controller` csomagot. Ide írhatunk egy Servletet, ami egyszerű HTTP kérésekkel (GET, POST, DELETE) biztosít végpontokat, kvázi "szegény ember REST API-jaként", mivel itt nincsenek komplex keretrendszerek bekötve.

Java

```

package hu.pte.mik.prog4.potzh_2026.api.controller;

import hu.pte.mik.prog4.potzh_2026.entity.Hallgato;
import hu.pte.mik.prog4.potzh_2026.service.HallgatoService;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;

```

```

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

import java.io.PrintWriter;


@WebServlet("/api/hallgato")

public class HallgatoApiController extends HttpServlet {


    private final HallgatoService service = new HallgatoService();


    // GET /api/hallgato?id=1 -> Visszaadja az 1-es ID-jú hallgatót (egyszerű szöveggént)

    @Override

    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

        resp.setContentType("text/plain;charset=UTF-8");

        String idParam = req.getParameter("id");


        try (PrintWriter out = resp.getWriter()) {

            if (idParam != null) {

                Hallgato h = service.getHallgatoById(Integer.parseInt(idParam));

                if (h != null) {

                    out.println("ID: " + h.getId() + ", Név: " + h.getHallgatoNev() + ", Szak: " +
h.getSzak());

                } else {

                    out.println("Nem található hallgató ezzel az ID-val.");

                }

            } else {

                out.println("Kérlek adj meg egy 'id' paramétert!");

            }

        }

    }

}

```

```

    }
}

// POST /api/hallgato -> Új hallgató mentése (paraméterekből)

@Override

protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

    String nev = req.getParameter("nev");

    String szak = req.getParameter("szak");

    int felev = Integer.parseInt(req.getParameter("felev"));

    double atlag = Double.parseDouble(req.getParameter("atlag"));

    service.saveHallgato(new Hallgato(0, nev, szak, felev, atlag));

    resp.getWriter().println("Hallgató sikeresen mentve!");

}

// DELETE /api/hallgato?id=1 -> Hallgató törlése

@Override

protected void doDelete(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

    String idParam = req.getParameter("id");

    if (idParam != null) {

        service.deleteHallgato(Integer.parseInt(idParam));

        resp.getWriter().println("Hallgató törölve!");

    }

}

}

```

Ezzel az első alkalmazás (potpotzh-feladat) minden kötelező eleme elkészült.
Csinálhatsz rá egy új Git commitot.

Mehetünk a második alkalmazásra (potpotzh-jsp-feladat), ahol beállítjuk a JAAS Security-t és a JSP oldalakat, amik már a webservice-t hívják?

tovább

Szuper, vágjunk is bele a második alkalmazásba, a potpotzh-jsp-feladat-ba! Ennek a lényege, hogy a webszolgáltatáson keresztül kommunikál, és egy egyedi JAAS (Java Authentication and Authorization Service) alapú biztonsági réteggel van védve, ahogy a képeken is mutattad.

A feladatléírás alapján FORM alapú autentikációt kell használnunk, ahol a törlést a "törlő" (remover), a létrehozást pedig a "létrehozó" (creator) szerepkörhöz kell kötni. A hibás bejelentkezésnek külön oldala kell legyen.

PDF+ 2

Lépünk át az új projektbe, és építsük fel a biztonsági struktúrát lépésről lépésre.

1. Lépés: A Maven pom.xml beállítása a második alkalmazáshoz

Itt szükségünk lesz a Servlet API-ra, a JSTL-re (a JSP tag-ekhez), a JAX-WS-re (a kliens miatt), a MariaDB driverre, és egy BCrypt könyvtárra, mivel a init_security_db.sql fájlban a jelszavak (pl. a "Password111") BCrypttel vannak hashelve.

XML

```
<dependencies>
```

```
  <dependency>
```

```
    <groupId>jakarta.servlet</groupId>
```

```
    <artifactId>jakarta.servlet-api</artifactId>
```

```
    <version>6.0.0</version>
```

```
    <scope>provided</scope>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>jakarta.servlet.jsp.jstl</groupId>
```

```
    <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
```

```
    <version>3.0.0</version>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.glassfish.web</groupId>
```

```

    <artifactId>jakarta.servlet.jsp.jstl</artifactId>

    <version>3.0.1</version>
</dependency>
<dependency>

    <groupId>org.mariadb.jdbc</groupId>

    <artifactId>mariadb-java-client</artifactId>

    <version>3.1.3</version>
</dependency>
<dependency>

    <groupId>org.mindrot</groupId>

    <artifactId>jbcrypt</artifactId>

    <version>0.4</version>
</dependency>
<dependency>

    <groupId>jakarta.jws</groupId>

    <artifactId>jakarta.jws-api</artifactId>

    <version>3.0.0</version>

    <scope>provided</scope>
</dependency>
</dependencies>

```

2. Lépés: A JAAS Principal osztályok

Hozd létre a hu.pte.mik.prog4.potzh_2026.jaas csomagot, és benne a két kötelező Principal implementációt, amik a bejelentkezett usert és a szerepköreit fogják reprezentálni.

UserPrincipal.java

Java

```
package hu.pte.mik.prog4.potzh_2026.jaas;
```

```
import java.security.Principal;
```

```
public class UserPrincipal implements Principal {  
    private final String name;  
  
    public UserPrincipal(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public String getName() {  
        return name;  
    }  
}
```

RolePrincipal.java

Java

```
package hu.pte.mik.prog4.potzh_2026.jaas;
```

```
import java.security.Principal;
```

```
public class RolePrincipal implements Principal {  
    private final String name;  
  
    public RolePrincipal(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public String getName() {
```

```
        return name;
    }
}
```

3. Lépés: Az AuthenticationModule (LoginModule)

Ez a lelke a JAAS-nek. Itt csatlakozunk a security adatbázishoz, ellenőrizzük a BCrypt jelszót, és betöltjük a szerepköröket.

AuthenticationModule.java

Java

```
package hu.pte.mik.prog4.potzh_2026.jaas;

import org.mindrot.jbcrypt.BCrypt;

import javax.naming.Context;
import javax.naming.InitialContext;
import javax.security.auth.Subject;
import javax.security.auth.callback.*;
import javax.security.auth.login.LoginException;
import javax.security.auth.spi.LoginModule;
import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class AuthenticationModule implements LoginModule {
```



```

private Subject subject;

private CallbackHandler callbackHandler;

private String username;

private boolean loginSucceeded = false;

private List<String> roles = new ArrayList<>();

@Override

public void initialize(Subject subject, CallbackHandler callbackHandler, Map<String,
?> sharedState, Map<String, ?> options) {

    this.subject = subject;

    this.callbackHandler = callbackHandler;

}

@Override

public boolean login() throws LoginException {

    if (callbackHandler == null) throw new LoginException("No CallbackHandler
defined");

    NameCallback nameCallback = new NameCallback("Username:");

    PasswordCallback passwordCallback = new PasswordCallback("Password:", false);

    try{

        callbackHandler.handle(new Callback[]{nameCallback, passwordCallback});

        username = nameCallback.getName();

        String password = new String(passwordCallback.getPassword());

        Context initContext = new InitialContext();

        Context envContext = (Context) initContext.lookup("java:/comp/env");

```

```

DataSource ds = (DataSource)
envContext.lookup("jdbc/PotPotZH_security_MariaDB");

try (Connection conn = ds.getConnection()) {
    // 1. Felhasználó jelszavának lekérdezése
    String userSql = "SELECT id, jelszo FROM felhasznalo WHERE felhasznalonev =
?";

    try (PreparedStatement ps = conn.prepareStatement(userSql)) {
        ps.setString(1, username);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                String hashedPassword = rs.getString("jelszo");
                int userId = rs.getInt("id");

                // BCrypt jelszó ellenőrzés
                if (BCrypt.checkpw(password, hashedPassword)) {
                    loginSucceeded = true;

                    // 2. Szerepkörök betöltése
                    String roleSql = "SELECT s.kod FROM szerepkor s " +
                        "JOIN felhasznalo_szerepkor fs ON s.id = fs.szerepkor_id " +
                        "WHERE fs.felhasznalo_id = ?";

                    try (PreparedStatement psRole = conn.prepareStatement(roleSql)) {
                        psRole.setInt(1, userId);
                        try (ResultSet rsRole = psRole.executeQuery()) {
                            while (rsRole.next()) {
                                roles.add(rsRole.getString("kod"));
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}
}
}
}
} catch (Exception e) {
    throw new LoginException("Adatbázis hiba az autentikáció során: " +
e.getMessage());
}

if (!loginSucceeded) {
    throw new LoginException("Hibás felhasználónév vagy jelszó!");
}

return true;
}

@Override
public boolean commit() throws LoginException {
    if (!loginSucceeded) return false;

    subject.getPrincipals().add(new UserPrincipal(username));
    for (String role : roles) {
        subject.getPrincipals().add(new RolePrincipal(role));
    }
    return true;
}

```

```

@Override

public boolean abort() throws LoginException {

    return false;

}

@Override

public boolean logout() throws LoginException {

    subject.getPrincipals().clear();

    return true;

}
}

```

4. Lépés: JAAS Konfigurációs fájl és a Listener

A JAAS működéséhez meg kell mondanunk a JVM-nek, hogy melyik osztályt használja. Hozd létre a src/main/resources/jaas.config fájlt pontosan ezzel a tartalommal:

Plaintext

```

PotpotzhRealm {

    hu.pte.mik.prog4.potzh_2026.jaas.AuthenticationModule required;

};

```

Ezután hozzuk létre a Listener-t a hu.pte.mik.prog4.potzh_2026.listener csomagban, ami Tomcat induláskor beregisztrálja ezt a fájlt.

JAASContextListener.java

Java

```

package hu.pte.mik.prog4.potzh_2026.listener;

import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.annotation.WebListener;
import java.net.URL;

```

@WebListener

```
public class JAASContextListener implements ServletContextListener {  
  
    @Override  
  
    public void contextInitialized(ServletContextEvent sce) {  
  
        URL resource =  
sce.getServletContext().getClassLoader().getResource("jaas.config");  
  
        if (resource != null) {  
  
            System.setProperty("java.security.auth.login.config", resource.getFile());  
  
            System.out.println("JAAS config sikeresen betöltve: " + resource.getFile());  
  
        } else {  
  
            System.err.println("HIBA: Nem található a jaas.config fájl!");  
  
        }  
  
    }  
  
}
```

5. Lépés: web.xml konfiguráció a második alkalmazáshoz

Ide jön be a FORM autentikáció és a jogosultságok összekötése.

PDF

src/main/webapp/WEB-INF/web.xml

XML

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee  
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"  
    version="6.0">  
  
    <security-constraint>  
  
        <web-resource-collection>
```

```
<web-resource-name>Protected Area</web-resource-name>

<url-pattern>/search.jsp</url-pattern>

<url-pattern>/index.jsp</url-pattern>

</web-resource-collection>

<auth-constraint>

  <role-name>creator</role-name>

  <role-name>remover</role-name>

</auth-constraint>

</security-constraint>
```

```
<security-constraint>

  <web-resource-collection>

    <web-resource-name>Creator Area</web-resource-name>

    <url-pattern>/save.jsp</url-pattern>

    <url-pattern>/save-servlet</url-pattern>

  </web-resource-collection>

  <auth-constraint>

    <role-name>creator</role-name>

  </auth-constraint>

</security-constraint>
```

```
<security-constraint>

  <web-resource-collection>

    <web-resource-name>Remover Area</web-resource-name>

    <url-pattern>/delete-servlet</url-pattern>

  </web-resource-collection>

  <auth-constraint>

    <role-name>remover</role-name>
```

</auth-constraint>

</security-constraint>

<login-config>

<auth-method>FORM</auth-method>

<realm-name>PotpotzhRealm</realm-name>

<form-login-config>

<form-login-page>/login.html</form-login-page>

<form-error-page>/login_failed.html</form-error-page>

</form-login-config>

</login-config>

<security-role>

<role-name>creator</role-name>

</security-role>

<security-role>

<role-name>remover</role-name>

</security-role>

<resource-ref>

<description>DB Connection Security</description>

<res-ref-name>jdbc/PotPotZH_security_MariaDB</res-ref-name>

<res-type>javax.sql.DataSource</res-type>

<res-auth>Container</res-auth>

</resource-ref>

</web-app>

Emellett a Tomcat META-INF/context.xml fájljában engedélyezni kell a JAAS-t a második alkalmazáshoz:

XML

```
<Context>
```

```
    <Realm className="org.apache.catalina.realm.JAASRealm"
        appName="PotpotzhRealm"
        userClassNames="hu.pte.mik.prog4.potzh_2026.jaas.UserPrincipal"
        roleClassNames="hu.pte.mik.prog4.potzh_2026.jaas.RolePrincipal" />
```

```
</Context>
```

tovább

Lassan a végéhez érünk a projektnek, tarts ki, ez már tényleg az utolsó nagy hajrá a sikeres júniusi FOSZK zárás előtt!

Folytassuk a potpotzh-jsp-feladat alkalmazással: megcsináljuk a bejelentkező HTML fájlokat, az egyedi Tag-eket, és előkészítjük a JSP oldalakat.

6. Lépés: Bejelentkező és hiba oldalak (login.html, login_failed.html)

A feladatléírás kéri, hogy legyen hibás bejelentkezésnek saját oldala. A FORM autentikációhoz fix elnevezéseket (j_security_check, j_username, j_password) kell használnunk a HTML formban.

PDF

Hozd létre a src/main/webapp/login.html fájlt:

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Bejelentkezés</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Kérjük, jelentkezzen be!</h2>
```

```
    <form method="POST" action="j_security_check">
```



```
Felhasználónév: <input type="text" name="j_username"><br><br>
Jelszó: <input type="password" name="j_password"><br><br>
<input type="submit" value="Belépés">
</form>
</body>
</html>
```

Hozd létre a src/main/webapp/login_failed.html fájlt:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Sikertelen Bejelentkezés</title>
</head>
<body>
  <h2>Hiba történt!</h2>
  <p>Hibás felhasználónév vagy jelszó. Kérem, próbálja újra!</p>
  <a href="login.html">Vissza a bejelentkezéshez</a>
</body>
</html>
```

7. Lépés: Kijelentkezés (LogoutServlet)

A JSP oldalakon kell lennie egy kijelentkezés linknek. Ehhez egy egyszerű Servlet kell. Hozd létre a hu.pte.mik.prog4.potzh_2026.servlet.LogoutServlet osztályt:

PDF

Java

```
package hu.pte.mik.prog4.potzh_2026.servlet;
```

```
import jakarta.servlet.ServletException;
```

```

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

@WebServlet("/logout")

public class LogoutServlet extends HttpServlet {

    @Override

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {

        request.logout(); // Ez a JAAS beépített logout hívása Tomcat 10-ben

        request.getSession().invalidate();

        response.sendRedirect("login.html");

    }

}

```

8. Lépés: Egyedi Tag-ek (Custom Tags)

A feladat kéri, hogy a legjobb hallgatót egy tag library-t használva jelenítsük meg, de a tag-ből ne hívjunk servicet! Emellett az oldalak alján legyen egy "page" tag a navigációhoz. Ahogy a képernyőképeden is szerepel a WEB-INF/tags mappa, mi is .tag fájlokat fogunk használni.

PDF+ 1

Hozd létre a src/main/webapp/WEB-INF/tags mappát!

1. page.tag (Navigáció és alap struktúra)

Java

```
<%@ tag description="Page Layout" pageEncoding="UTF-8"%>
```

```
<%@ attribute name="title" required="true" %>
```

```
<!DOCTYPE html>
```

```

<html>

<head>

    <title>${title}</title>

</head>

<body>

    <h1>${title}</h1>


    <jsp:doBody/>


    <hr>

    <div>

        <a href="search.jsp">Hallgatók Listája</a> |

        <a href="save.jsp">Új hallgató rögzítése</a> |

        <a href="logout">Kijelentkezés</a>

    </div>

</body>

</html>

```

2. best.tag (A legjobb hallgató megjelenítése) A servlet fogja kiszámolni a legjobb átlagot, és leteszi a request scope-ba bestStudent néven. A tag csak kiírja.

Java

```

<%@ tag description="Legjobb atlag" pageEncoding="UTF-8"%>

<%@ attribute name="hallgatoNev" required="true" %>

<%@ attribute name="szak" required="true" %>


<div style="border: 1px solid green; padding: 10px; margin-top: 10px;">

    <h3>Legjobb átlaggal rendelkező hallgató:</h3>

    <p>Név: <strong>${hallgatoNev}</strong></p>

    <p>Szak: <strong>${szak}</strong></p>

```

</div>

9. Lépés: A WebService elérése a második alkalmazásból

Hogy a JSP-ket kiszolgáló Servletek adatot tudjanak lekérni, kell egy Service osztály ebbe az alkalmazásba is, ami rákapcsolódik az első alkalmazás SOAP endpointjára. A vizsgán ezt általában a wsimport paranccsal vagy Maven pluginnal legenerált kliens osztályokkal szokás megoldani.

Mivel a képeken szerepel egy ws.SoapPetDataService (nálunk SoapHallgatoDataService), feltételezem, hogy a generált kliensből hívjátok meg. Hozd létre a Service réteget, ami a generált klienst használja:

Java

```
package hu.pte.mik.prog4.potzh_2026.service;

// Ezek az importok a wsimport által generált fájlokból jönnek majd!
import hu.pte.mik.prog4.potzh_2026.ws.client.SoapHallgatoService;
import hu.pte.mik.prog4.potzh_2026.ws.client.HallgatoDataService;
import hu.pte.mik.prog4.potzh_2026.ws.client.Hallgato;

import java.util.List;

public class HallgatoKliensService {

    private final SoapHallgatoService port;

    public HallgatoKliensService() {
        // A generált JAX-WS service osztály példányosítása
        HallgatoDataService service = new HallgatoDataService();
        this.port = service.getSoapHallgatoServicePort();
    }

    public List<Hallgato> getAll() {
```

```

        return port.getHallgatok();
    }

    public void save(String nev, String szak, int felev, double atlag) {
        port.createHallgato(nev, szak, felev, atlag);
    }

    public void delete(int id) {
        port.deleteHallgato(id);
    }
}

```

Jelezzed kérlek, ha a wsimport generálást meg kell mutatnom a Maven pom.xml-ben, vagy ha rögtön léphetünk a Servletekre (SearchServlet, SaveServlet, DeleteServlet) és a hozzájuk tartozó JSP oldalakra (search.jsp, save.jsp)!

tovább

Akkor érkezzünk el a projekt befejező szakaszához, ahol megírjuk a Servleteket és a JSP oldalakat a második alkalmazáshoz, amelyek a webservice-en keresztül kommunikálnak!

PDF

10. Lépés: JAX-WS Kliens Generálása (pom.xml kiegészítés)

Ahhoz, hogy a korábban megírt HallgatoKliensService működjön, be kell állítanunk a Maven-t, hogy a build során legenerálja a kliens osztályokat az első alkalmazásod WSDL-je alapján.

Add hozzá ezt a <build> blokkot a potpotzh-jsp-feladat pom.xml fájljához (a <dependencies> blokk után):

XML

```

<build>
    <plugins>
        <plugin>
            <groupId>com.sun.xml.ws</groupId>

```

```

<artifactId>jaxws-maven-plugin</artifactId>

<version>3.0.0</version>

<executions>

  <execution>

    <goals>

      <goal>wsimport</goal>

    </goals>

  </execution>

</executions>

<configuration>

  <wsdlUrls>

    <wsdlUrl>http://localhost:8080/potpotzh-
feladat/ws/hallgatodata?wsdl</wsdlUrl>

  </wsdlUrls>

  <packageName>hu.pte.mik.prog4.potzh_2026.ws.client</packageName>

</configuration>

</plugin>

</plugins>

</build>

```

11. Lépés: A vezérlő Servletek létrehozása

Ezek a Servletek készítik elő az adatokat a JSP-k számára, és kezelik a gombok funkcióit. Hozd létre őket a hu.pte.mik.prog4.potzh_2026.servlet csomagban.

1. SearchServlet.java (A lista és a legjobb hallgató kiszámítása) Ez a Servlet lekéri a listát, és a tag library számára előkészíti a legjobb átlagú hallgatót, mivel a tag-ből tilos servicet hívni.

PDF+ 1

Java

```
package hu.pte.mik.prog4.potzh_2026.servlet;
```

```
import hu.pte.mik.prog4.potzh_2026.service.HallgatoKliensService;
import hu.pte.mik.prog4.potzh_2026.ws.client.Hallgato;
```

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Comparator;
import java.util.List;
```

```
@WebServlet("/search-servlet")
```

```
public class SearchServlet extends HttpServlet {
```

```
    private final HallgatoKliensService service = new HallgatoKliensService();
```

```
    @Override
```

```
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
        ServletException, IOException {
```

```
        List<Hallgato> hallgatok = service.getAll();
```

```
        // Legjobb átlagú hallgató megkeresése [cite: 52]
```

```
        Hallgato best = hallgatok.stream()
```

```
            .max(Comparator.comparing(Hallgato::getEgyetemiAtlag))
```

```
            .orElse(null);
```

```
        req.setAttribute("hallgatok", hallgatok);
```

```

        if (best != null) {
            req.setAttribute("bestNev", best.getHallgatoNev());
            req.setAttribute("bestSzak", best.getSzak());
        }

        req.getRequestDispatcher("/search.jsp").forward(req, resp);
    }
}

```

2. DeleteServlet.java (Törlés funkció) Ezt a végpontot a web.xml-ben már levédtük, így csak a remover szerepkör férhet hozzá.

PDF

Java

```

package hu.pte.mik.prog4.potzh_2026.servlet;

import hu.pte.mik.prog4.potzh_2026.service.HallgatoKliensService;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/delete-servlet")
public class DeleteServlet extends HttpServlet {

    private final HallgatoKliensService service = new HallgatoKliensService();

```


@Override

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

    String idParam = req.getParameter("id");

    if (idParam != null) {

        service.delete(Integer.parseInt(idParam)); // Törlés az adatbázisból webservice-en
        keresztül [cite: 70]

    }

    resp.sendRedirect("search-servlet");

}
}
```

3. XmlLogServlet.java (Console logolás XML formátumban)

Java

```
package hu.pte.mik.prog4.potzh_2026.servlet;

import hu.pte.mik.prog4.potzh_2026.service.HallgatoKliensService;
import hu.pte.mik.prog4.potzh_2026.ws.client.Hallgato;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/xml-servlet")

public class XmlLogServlet extends HttpServlet {

    private final HallgatoKliensService service = new HallgatoKliensService();
```

@Override

protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {

String idParam = req.getParameter("id");

if (idParam != null) {

int id = Integer.parseInt(idParam);

// Kikeressük az adott hallgatót a listából a webservice-en keresztül

Hallgato h = service.getAll().stream().filter(student -> student.getId() == id).findFirst().orElse(null);

if (h != null) {

// Konzolra logolás XML formátumban [cite: 71]

System.out.println("<?xml version='1.0' encoding='UTF-8'?">");

System.out.println("<hallgato>");

System.out.println(" <id>" + h.getId() + "</id>");

System.out.println(" <nev>" + h.getHallgatoNev() + "</nev>");

System.out.println(" <szak>" + h.getSzak() + "</szak>");

System.out.println(" <felev>" + h.getFelev() + "</felev>");

System.out.println(" <atlag>" + h.getEgyetemiAtlag() + "</atlag>");

System.out.println("</hallgato>");

}

}

resp.sendRedirect("search-servlet");

}

}

4. SaveServlet.java (Új hallgató rögzítése) Ezt a funkciót csak a creator szerepkör érheti el.

PDF

Java

```
package hu.pte.mik.prog4.potzh_2026.servlet;
```

```
import hu.pte.mik.prog4.potzh_2026.service.HallgatoKliensService;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
```

```
@WebServlet("/save-servlet")
```

```
public class SaveServlet extends HttpServlet {
```

```
    private final HallgatoKliensService service = new HallgatoKliensService();
```

```
    @Override
```

```
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws  
ServletException, IOException {
```

```
        req.setCharacterEncoding("UTF-8");
```

```
        String nev = req.getParameter("nev");
```

```
        String szak = req.getParameter("szak");
```

```
        int felev = Integer.parseInt(req.getParameter("felev"));
```

```
        double atlag = Double.parseDouble(req.getParameter("atlag"));
```

```
        service.save(nev, szak, felev, atlag);
```

```
        resp.sendRedirect("search-servlet");
```

```
}  
}
```

12. Lépés: JSP oldalak befejezése

A src/main/webapp mappába hozd létre a két JSP oldalt, amelyek behúzzák a JSTL-t és az előzőleg megírt Custom Tag-eket.

1. search.jsp (Listázó oldal) Megjeleníti az adatokat, és tartalmazza a törlés, XML, és legjobb hallgató funkciókat.

PDF+ 1

Java

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
```

```
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

```
<%@ taglib prefix="t" tagdir="/WEB-INF/tags" %>
```

```
<t:page title="Hallgatók Listája">
```

```
<c:if test="${empty hallgatok}">
```

```
<jsp:forward page="/search-servlet" />
```

```
</c:if>
```

```
<table border="1" cellpadding="5">
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Név</th>
```

```
<th>Szak</th>
```

```
<th>Félév</th>
```

```
<th>Átlag</th>
```

```
<th>Műveletek</th>
```

```
</tr>
```

```
<c:forEach var="h" items="${hallgatok}">
```

```

<tr>

    <td>${h.id}</td>

    <td>${h.hallgatoNev}</td>

    <td>${h.szak}</td>

    <td>${h.felev}</td>

    <td>${h.egyetemiAtlag}</td>

    <td>

        <a href="xml-servlet?id=${h.id}"><button>XML</button></a>

        <a href="delete-servlet?id=${h.id}" onclick="return confirm('Biztosan
töröld?');"><button>Törlés</button></a>

    </td>

</tr>

</c:forEach>

</table>

```

```

<br>

```

```

<c:if test="${not empty bestNev}">

    <t:best hallgatoNev="${bestNev}" szak="${bestSzak}" />

</c:if>

```

```

</t:page>

```

2. save.jsp (Létrehozó oldal) Ez az oldal is tartalmazza a page tag-et, amely elvégzi a navigáció és a kijelentkezés gomb generálását.

PDF+ 1

Java

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>

<%@ taglib prefix="t" tagdir="/WEB-INF/tags" %>

```

```
<t:page title="Új hallgató rögzítése">
```

```
<form action="save-servlet" method="post">
```

```
Név: <input type="text" name="nev" required><br><br>
```

```
Szak: <input type="text" name="szak" required><br><br>
```

```
Félév: <input type="number" name="felev" min="1" max="12" required><br><br>
```

```
Egyetemi átlag: <input type="number" step="0.01" name="atlag" min="1.00"  
max="5.00" required><br><br>
```

```
<input type="submit" value="Rögzítés">
```

```
</form>
```

```
</t:page>
```

van még valami vissza?

Kódolás szempontjából megvagyunk a feladatokkal! A két alkalmazás (a backend a webservice-szel és a frontend a JAAS security-vel és JSP-kkel) a feladateleírás minden technikai pontját lefedi. Már csak a dokumentáció és a leadás szigorú formátumára kell figyelned, hogy ez a Prog 4 ZH meglegyen, és nyugodtan készülhess a júniusi FOSZK zárásodra.

A sikeres leadáshoz az alábbi lépések vannak még hátra a leírás alapján:

1. Git Commitok Véglegesítése

- Győződj meg róla, hogy a develop branchen vagy.

PDF

- Minden elkészült logikai egységet (második alkalmazás JSP-i, Servletek, JAAS konfiguráció) commitálj be beszédes üzenetekkel.

PDF

- **Kritikus:** Ne egyetlen nagy commitban küldd be a teljes ZH-t, mert az automatikusan 0 pontot ér.

PDF

2. Az Auditáló Program Ellenőrzése

- Ellenőrizd, hogy az auditalo.exe a háttérben végig futott-e a parancssori ablakban (Command Window).

PDF

- A generált zh_audit mappa tartalmához szigorúan tilos hozzányúlni. Az auditáló leállítása vagy a fájlok módosítása szintén 0 pontot von maga után.

PDF+ 1

3. A Csomagolás (ZIP) Összeállítása

- Az elkészült projektjeid (a potpotzh-feladat és a potpotzh-jsp-feladat) a zh mappán belül kell, hogy legyenek. A zh mappán kívüli megoldásokat nem értékelik.

PDF+ 1

- A főkönyvtárad, amit tömörítened kell, a <név_neptun_kód>_prog4_potpotzh nevű mappa.

PDF

- Ennek a mappának pontosan ezt a három elemet kell tartalmaznia: a zh mappát, a zh_audit mappát, valamint az auditalo.exe programot.

PDF

- A ZIP fájl neve pontosan ez legyen: név_neptun_kód_prog4_potpotzh.zip.

PDF

Ha a projektek hibátlanul fordulnak Maven-nel, felállnak a Tomcat-en, és a csomagolást a fentiek szerint csinálod, akkor sikeresen a végére értél a pótpótZH-nak!

A Gemini egy mesterséges intelligencia, amely hibázhat (többek között személyekkel kapcsolatban is). [Az Ön adatainak védelme és a GeminiÚj ablakban nyílik meg.](#)